



Problems:

- 1) We need to specify many dependencies for getting different Jars belongs to different technologies. (with pom.xml)
=> What the Jars to download
- 2) We need to check Version Compatibility and need to specify the Version Compatibility (with pom.xml)
=> What version of Jars to download
- 3) Complexity in Configuring many Beans in Bean Configuration class
- 4) Hard-Coding Bean Properties in Bean Configuration class

How SpringBoot Solves the Above Problems.

1) Starters.

- Gets All the Dependent Compatible Jars with given starter name.
 - ✓ spring-boot-starter-jdbc
 - ✓ spring-boot-starter-data-jpa
 - ✓ spring-boot-starter-data-mongodb
 - ✓ spring-boot-starter-web
 - ✓ spring-boot-starter-thymeleaf
 - ✓ spring-boot-starter-security
 - ✓ spring-boot-starter-mail
 - ✓ spring-boot-starter-test
- and Many more

2) AutoConfiguration

- Most of the Beans will be configured by the Spring Container automatically.
- Beans will be configured conditionally based availability of Classes in the Classpath.

3) Centralizing the Configuration Data

- You can centralize the Configuration Data/Parameters in Property Files or YAML Files



boot-data-access-jdbc-demo2: Spring Boot and JDBC

boot-data-access-jdbc-demo2: Working Steps:

A) Setup Database

```
create database myjlcdb5;  
use myjlcdb5;  
create table mycustomers(  
  cid int primary key,  
  cname char(10),  
  email char(10),  
  phone long,  
  city char(10)  
);
```

B) Create the maven Project.

- 1) Open the Eclipse required Workspace
- 2) Select New -> Maven Project.
- 3) Check the checkbox for create Simple Project (skip the Archetype selection) and Click on Next Button.
- 4) Provide the following
Group Id: **com.jlcindia.springboot**
Artifact Id : **boot-data-access-jdbc-demo2**
Version : 1.0
packaging : jar
Name : **boot-data-access-jdbc-demo2**

and Click On finish button.

- 5) Remove the following two Source Folders.

- a. src/test/java
- b. src/test/resources

C) Start the Example Code.

- 1) Add the spring-boot-maven-plugin in pom.xml

```
<plugin>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-maven-plugin</artifactId>  
</plugin>
```



2) Update pom.xml by adding the following dependencies.

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
</dependency>

</dependencies>
```

3) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

Select the Project, right click and then select Maven -> Update Project.

4) Create **application.properties** under **src/main/resources**

5) Add the following properties in application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

#Log Properties

```
logging.level.root=DEBUG
logging.pattern.console=%-5level %logger{36} - %msg%n
```



- 6) Create the Package called com.jlcindia.springboot
- 7) Write JLCAppConfig.java (Bean Configuration class)
 - No Beans to Configure

```
@SpringBootApplication
public class JLCAppConfig {

}
```

- 8) Write the following Files (**Copy from Demo1**)
 - Customer.java
 - CustomerRowMapper.java
- 9) Write the DAO and its Implementation class (**Copy from Demo1**)
 - CustomerDAO.java
 - CustomerDAOImpl.java
- 10) Write Boot Main Class - **MyBootApplication** and Run it.

Run As => Spring Boot Application

Spring -JDBC Example with Maven

boot-data-access-jdbc-demo2: Files required

1. MyBootApplication.java	Newly Added in Demo2
2. CustomerDAO.java	Same As Demo1
3. CustomerDAOImpl.java	Same As Demo1
4. Customer.java	Same As Demo1
5. CustomerRowMapper.java	Same As Demo1
6. JLCAppConfig.java	Updated in Demo2
7. Application.properties	Newly Added in Demo2
8. pom.xml	Updated in Demo2



1) MyBootApplication.java

```
package com.jlcindia.spring;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust = new Customer(106, "hai", "hai@jlc", 12345, "Blore");
        custDAO.addCustomer(cust);

        // 2.get All Customers
        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");

        // 3.Get Customers By City
        List<Customer> mylist2 = custDAO.getCustomersByCity("Blore");
        mylist2.forEach(mycust -> System.out.println(mycust));

        System.out.println("Done!!!");
    }

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```



6)JLAppConfig.java

```
package com.jlcindia.spring;

import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class JLAppConfig {
}
```

7)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.maximum-pool-size=100
spring.datasource.hikari.idle-timeout=20000
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

8)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>boot-data-access-jdbc-demo2</artifactId>
<version>1.0</version>
<name>boot-data-access-jdbc-demo2</name>
<description>This is My First Spring Boot Application</description>
<url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Interview Questions

- Q1)** What is Spring Boot?
- Q2)** What are the Spring Boot features discussed till now?
- Q3)** What is use of Spring Boot Starters?
- Q4)** What is Auto Configuration?
- Q5)** Can I ask the Spring Boot to Auto-Configure CustomerDAOImpl?
- Q6)** Is it required to me to Configure CustomerDAOImpl in Bean Configuration class?
- Q7)** How to Configure the Beans which I am writing In My applications?
- Q8)** How Spring Boot will decide which bean to Auto-Configure?
- Q9)** What is the use of Property File / YAML File?
- Q10)** Can I have both files (properties and yaml)?
- Q11)** How Can I Spring Application in Oracle?
- Q12)** What are various Third Party DataSources you know?
- Q13)** Can I configure two Databases related Properties in One Boot Application?
- Q14)** How Can I exclude the Jars which are downloaded by Spring Boot Starters?
- Q15)** Can I write my own Spring Boot Starters?